

从正规表达式到最小 DFA

词法分析自动机转换完整学习笔记

RE $\rightarrow$ Thompson NFA $\rightarrow$ Subset DFA $\rightarrow$ Minimized DFA

2026 年 6 月 27 日

目录

阅读导引	3
1 基础概念	3
1.1 正规表达式	3
1.2 NFA 与 DFA	4
2 Thompson 算法：RE 到 NFA	4
2.1 算法思想	4
2.2 基本构造规则	5
2.3 Thompson 算法步骤	5
2.4 复杂度分析	5
3 完整示例：构造 Thompson NFA	6
4 子集构造法：NFA 到 DFA	6
4.1 核心思想	6
4.2 算法流程	7
4.3 复杂度分析	7
5 完整示例：子集构造 DFA	7
6 DFA 最小化	8
6.1 为什么要最小化	8
6.2 划分细化算法	8

目录	2
6.3 复杂度分析	9
7 完整示例：DFA 最小化	9
8 工程实现提示	10
8.1 从多个 token 规则构造词法分析器	10
8.2 常见易错点	11
9 练习题与解答	11
复习总结	13

阅读导引

词法分析器的核心任务是把源代码字符流切分为 token。一个常见的工程路径是：用正规表达式描述 token 模式，把正规表达式机械地转成 NFA，再把 NFA 转成 DFA，最后最小化 DFA 并生成高效识别代码。



贯穿示例

本文使用正规表达式

$$R = (a|b)^*abb$$

作为完整示例。它表示所有以 abb 结尾、前面可以有任意个 a 或 b 的字符串，例如 abb、aabb、bababb。

1 基础概念

1.1 正规表达式

正规表达式

正规表达式是一种描述正规语言的代数记号。它用有限规则描述一类字符串集合，是词法规则最常见的表示方式。

常见操作：

符号	名称	含义
a	字符	只匹配字符 a
ε	空串	匹配长度为 0 的字符串
$\emptyset$	空集	不匹配任何字符串
$r s$	并 / 选择	匹配 r 或 s
rs	连接	先匹配 r ，再匹配 s
r^*	Kleene 闭包	匹配 0 次或多次 r

r^+	正闭包	匹配 1 次或多次 r , 可写成 rr^*
$r?$	可选	匹配 0 次或 1 次 r , 可写成 $r \varepsilon$

运算优先级通常为: 闭包最高, 连接其次, 并最低。因此 $(a|b)^*abb$ 的结构是:

$$(((a|b)^*)a)b)b.$$

1.2 NFA 与 DFA

NFA

非确定有限自动机 NFA 是五元组

$$N = (Q, \Sigma, \delta, q_0, F).$$

Q 是状态集合, Σ 是输入字母表, δ 是转移函数, q_0 是初态, F 是终态集合。NFA 的一个状态在同一输入字符上可以有多个后继, 也可以有 ε 转移。

DFA

确定有限自动机 DFA 也是五元组

$$D = (Q, \Sigma, \delta, q_0, F),$$

但每个状态在每个输入字符上最多有一个确定后继, 且没有 ε 转移。DFA 运行时只需维护一个当前状态。

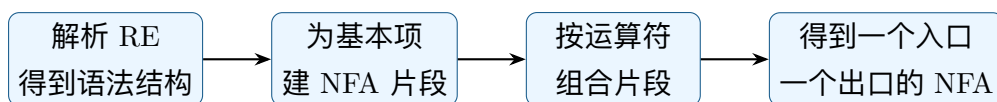
通俗比较:

- NFA 像“同时尝试多条路”。读一个字符后, 可能分裂到多个状态。
- DFA 像“始终只站在一个确定位置”。每读一个字符, 只做一次表查询。
- Thompson 算法让 RE 到 NFA 很容易; 子集构造把 NFA 的多条可能路径打包成 DFA 的一个状态。

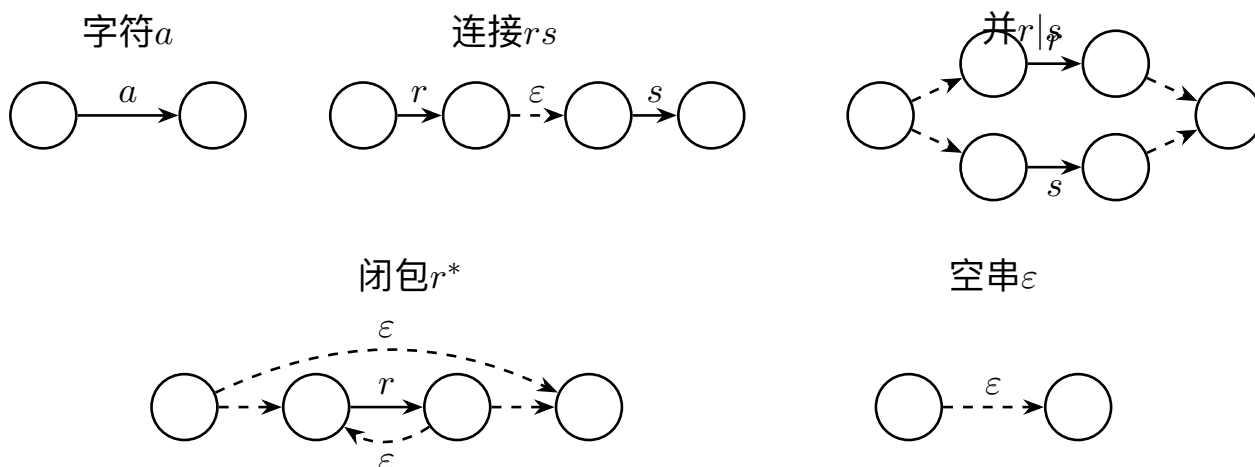
2 Thompson 算法: RE 到 NFA

2.1 算法思想

Thompson 构造把正规表达式看作由小表达式组合而成。每个子表达式都构造一个 NFA 片段, 片段有且只有一个入口状态和一个出口状态; 然后根据并、连接、闭包等运算把片段拼起来。



2.2 基本构造规则



为什么 Thompson 构造好用

它的每个片段接口统一：一个入口、一个出口。连接只需把前一个出口接到后一个入口；并和闭包只需额外加少量 ϵ 边。因此算法可以按语法树后序遍历线性构造。

2.3 Thompson 算法步骤

1. 给正规表达式补充显式连接符，例如把 $(a|b)^*abb$ 看成 $(a|b)^* \cdot a \cdot b \cdot b$ 。
2. 按优先级或语法树解析表达式。
3. 后序处理语法树：
 - 遇到字符、 ϵ 、 $\emptyset$ ，构造基本片段。
 - 遇到连接、并、闭包，把子片段按规则组合。
4. 最终得到一个 NFA，初态为总片段入口，终态为总片段出口。

2.4 复杂度分析

设正规表达式长度为 m 。

- 状态数： $O(m)$ 。每个基本符号和运算符只引入常数个状态。

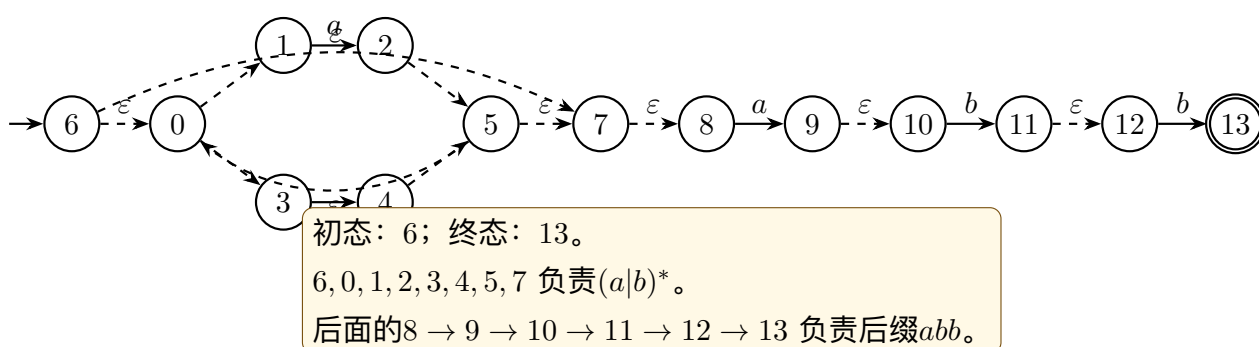
- 转移数： $O(m)$ 。每个构造规则只加入常数条边。
- 构造时间： $O(m)$ 。若已完成词法/语法解析，后序遍历即可线性构造。
- 空间复杂度： $O(m)$ 。

3 完整示例：构造 Thompson NFA

对

$$R = (a|b)^*abb$$

构造 Thompson NFA。先构造 $a|b$ ，再加闭包 $(a|b)^*$ ，最后依次连接 a, b, b 。



从这张图可以看出， ϵ 边负责“自动选择路径”或“重复/跳过”。例如状态 6 可通过 ϵ 直接到状态 7，表示 $(a|b)^*$ 可以重复 0 次；状态 5 可通过 ϵ 回到状态 0，表示继续重复。

4 子集构造法：NFA 到 DFA

4.1 核心思想

DFA 运行时只能处在一个状态，而 NFA 可能同时处在多个状态。子集构造法的关键是：把 NFA 的一个状态集合当作 DFA 的一个状态。

epsilon-closure

$\epsilon\text{-closure}(S)$ 表示从状态集合 S 出发，只沿 ϵ 边能到达的所有状态，包括 S 本身。

move

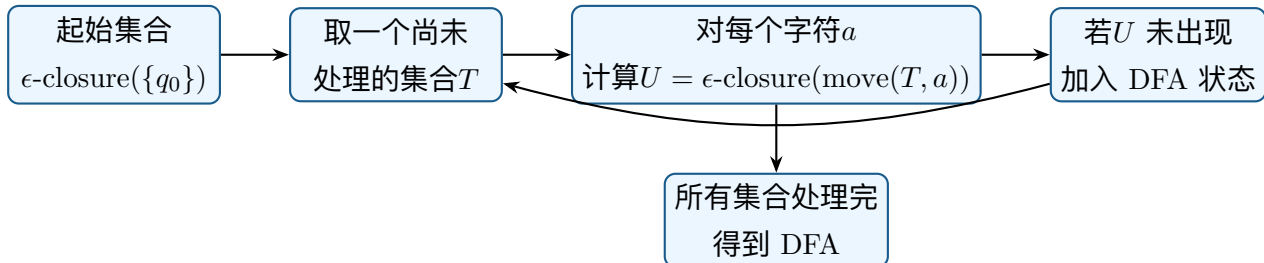
$\text{move}(S, a)$ 表示从状态集合 S 中任意状态出发，沿字符 a 边能到达的状态集合。

子集构造的转移公式是：

$$T' = \epsilon\text{-closure}(\text{move}(T, a)).$$

其中 T 是一个 DFA 状态，也就是一组 NFA 状态； a 是输入字符。

4.2 算法流程



伪代码：

```

Dstates = { epsilon-closure({NFA_start}) }
while exists unmarked T in Dstates:
    mark T
    for each input symbol a:
        U = epsilon-closure(move(T, a))
        if U is not in Dstates:
            add U to Dstates
        Dtran[T, a] = U
  
```

若某个 DFA 状态集合包含 NFA 终态，则该 DFA 状态就是终态。

4.3 复杂度分析

设 NFA 有 n 个状态，字母表大小为 $|\Sigma|$ 。

- 最坏 DFA 状态数： 2^n 。因为 NFA 状态集合最多有 2^n 种。
- 转移计算：每个可达子集要对每个输入字符计算一次 move 和 closure。
- 朴素实现时间：可写为 $O(2^n |\Sigma| (n + e))$ ，其中 e 为 NFA 转移数。
- 实际情况：词法分析中的正则通常结构简单，可达子集远少于 2^n ，因此工程上可接受。

5 完整示例：子集构造 DFA

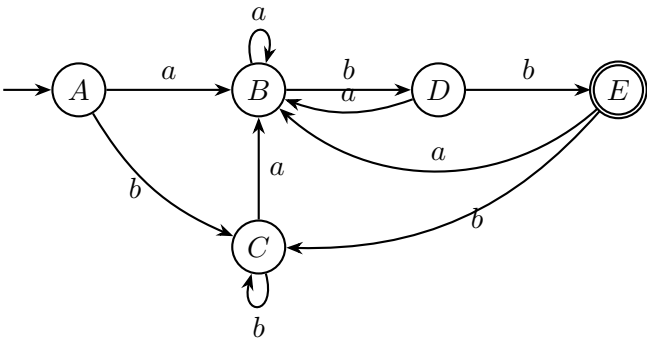
对上一节的 NFA，字母表 $\Sigma = \{a, b\}$ 。从初态 6 开始：

$$A = \epsilon\text{-closure}(\{6\}) = \{0, 1, 3, 6, 7, 8\}.$$

逐步计算得到 DFA 状态：

DFA 状态	NFA 状态集合	输入 a	输入 b
A	$\{0, 1, 3, 6, 7, 8\}$	B	C
B	$\{0, 1, 2, 3, 5, 7, 8, 9, 10\}$	B	D
C	$\{0, 1, 3, 4, 5, 7, 8\}$	B	C
D	$\{0, 1, 3, 4, 5, 7, 8, 11, 12\}$	B	E
E	$\{0, 1, 3, 4, 5, 7, 8, 13\}$	B	C

状态 E 包含 NFA 终态 13，因此 E 是 DFA 终态。



这个 DFA 已经没有 ϵ 转移，也不会在同一字符上分裂到多个状态。但它还不是最小的，因为 A 和 C 行为完全相同。

6 DFA 最小化

6.1 为什么要最小化

DFA 的执行速度主要由“当前状态 + 输入字符”的转移表查询决定；状态越少，表越小，生成的词法分析器通常越紧凑。DFA 最小化就是把识别同一语言且行为不可区分的状态合并。

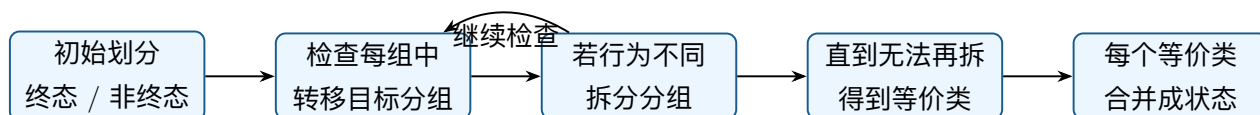
等价状态

两个 DFA 状态 p, q 等价，是指从 p 和 q 出发，读任意后缀字符串 w ，最终接受/拒绝结果都相同。这样的状态可以合并。

6.2 划分细化算法

最常用的直观算法是 partition refinement：

1. 初始划分为两组：终态集合 F 与非终态集合 $Q - F$ 。
2. 对每个分组，检查其中状态在每个输入字符上的转移目标属于哪个分组。
3. 若同一组中存在状态在某字符上跳到不同分组，则拆分该组。
4. 重复细化，直到所有分组都不能再拆。
5. 每个分组合并为最小 DFA 的一个状态。



6.3 复杂度分析

设 DFA 有 k 个状态，字母表大小为 $|\Sigma|$ 。

- 朴素表填法：通常为 $O(k^2|\Sigma|)$ 。
- 普通划分细化：实现不同，常见朴素版本可到 $O(k^2|\Sigma|)$ 。
- Hopcroft 算法：经典最优量级之一，为 $O(|\Sigma|k \log k)$ 。
- 空间复杂度：至少需要保存转移表 $O(k|\Sigma|)$ ，以及划分或标记信息。

7 完整示例：DFA 最小化

当前 DFA 状态为 A, B, C, D, E ，其中 E 是终态。

初始划分：

$$P_0 = \{\{E\}, \{A, B, C, D\}\}.$$

检查非终态组 $\{A, B, C, D\}$ ：

状态	输入 a 的目标组	输入 b 的目标组	结论
A	非终态组	非终态组	与 C 相同行为
B	非终态组	非终态组	暂时可与 A, C 同组，但后续需再细分
C	非终态组	非终态组	与 A 相同行为
D	非终态组	终态组	必须单独分出

第一次细分：

$$P_1 = \{\{E\}, \{D\}, \{A, B, C\}\}.$$

继续检查 $\{A, B, C\}$ ：

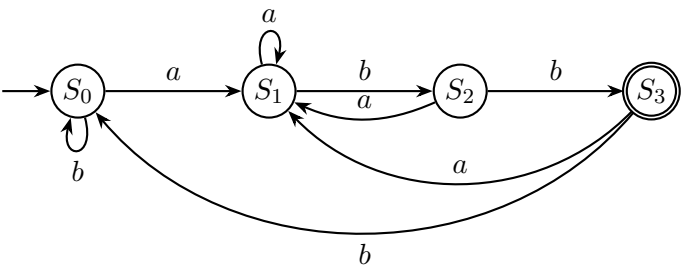
- A 在 a 上到 B , 在 b 上到 C , 都在 $\{A, B, C\}$ 。
- C 在 a 上到 B , 在 b 上到 C , 都在 $\{A, B, C\}$ 。
- B 在 a 上到 B , 在 b 上到 D , 而 D 已经是单独分组。

因此:

$$P_2 = \{\{E\}, \{D\}, \{B\}, \{A, C\}\}.$$

此时无法继续细分。合并 A 和 C , 得到最小 DFA:

最小状态	来源	输入 a	输入 b
S_0	$\{A, C\}$	S_1	S_0
S_1	$\{B\}$	S_1	S_2
S_2	$\{D\}$	S_1	S_3
S_3	$\{E\}$	S_1	S_0



这个最小 DFA 的含义非常直观:

- S_0 : 当前尚未匹配到有用后缀, 或当前后缀相当于“无前缀匹配”。
- S_1 : 当前最长匹配后缀是 a 。
- S_2 : 当前最长匹配后缀是 ab 。
- S_3 : 当前最长匹配后缀是 abb , 接受。

8 工程实现提示

8.1 从多个 token 规则构造词法分析器

真实词法分析器通常不只处理一个正规表达式, 而是处理多条 token 规则:

$$R_1, R_2, \dots, R_n.$$

常见做法:

1. 分别为每个 R_i 构造 Thompson NFA。
2. 新建一个总初态，用 ϵ 边连到每个规则 NFA 的初态。
3. 每个规则的终态标注 token 类型和优先级。
4. 子集构造后，若一个 DFA 状态包含多个 NFA 终态，按优先级选 token。
5. 扫描输入时采用最长匹配原则：不断前进，记录最近一次到达接受状态的位置和 token。

8.2 常见易错点

- 忘记在子集构造中每一步都做 ϵ -closure。
- 把 NFA 的“多个可能状态”误当作 DFA 的“多个当前状态”。DFA 状态本身就是一个集合。
- 最小化前没有删除不可达状态。严格流程通常先删除从初态不可达的状态，再做最小化。
- 只用“是否终态”判断等价状态。终态/非终态只是初始划分，后续还要看所有字符的转移行为。
- 在真实 lexer 中忽略最长匹配和规则优先级。

9 练习题与解答

题 1：为什么 Thompson NFA 通常有很多 epsilon 转移？

解答：因为 Thompson 构造要保持“每个片段一个入口、一个出口”的统一接口。并、闭包、连接都通过 ϵ 边把片段组合起来。这样构造规则简单，代价是 NFA 中出现大量 ϵ 转移，后续需要用 ϵ -closure 处理。

题 2：对子集构造，为什么 DFA 状态是 NFA 状态集合？

解答：NFA 在读入一个前缀后，可能同时处在多个状态。为了用 DFA 模拟 NFA，必须把这些“所有可能状态”整体记录下来。这个集合就是 DFA 的一个状态。

题 3：在本文示例中，初始 DFA 状态 A 是什么？

解答：

$$A = \epsilon\text{-closure}(\{6\}) = \{0, 1, 3, 6, 7, 8\}.$$

它包含初态 6 以及只沿 ϵ 边能到达的所有状态。

题 4：为什么状态 E 是 DFA 接受状态？

解答：因为

$$E = \{0, 1, 3, 4, 5, 7, 8, 13\}$$

包含 Thompson NFA 的终态 13。子集构造中，只要某个 DFA 状态集合包含任一 NFA 终态，该 DFA 状态就是接受状态。

题 5：为什么示例 DFA 中 A 和 C 可以合并？

解答：A 和 C 都不是终态，并且它们对每个输入字符的行为完全相同：

$$A \xrightarrow{a} B, \quad A \xrightarrow{b} C,$$

$$C \xrightarrow{a} B, \quad C \xrightarrow{b} C.$$

读任意后缀时，从 A 和 C 出发的接受/拒绝结果都相同，因此二者等价。

题 6：最小 DFA 有几个状态？分别表示什么？

解答：对 $(a|b)^*abb$ ，最小 DFA 有 4 个状态：

- S_0 ：当前后缀没有匹配到模式前缀。
- S_1 ：当前后缀是 a 。
- S_2 ：当前后缀是 ab 。
- S_3 ：当前后缀是 abb ，接受。

题 7：如果输入串是 bababb，最小 DFA 如何运行？

解答：

$$S_0 \xrightarrow{b} S_0 \xrightarrow{a} S_1 \xrightarrow{b} S_2 \xrightarrow{a} S_1 \xrightarrow{b} S_2 \xrightarrow{b} S_3.$$

最终到达接受状态 S_3 ，所以 bababb 被接受。

题 8：如果输入串是 ababa，是否接受？

解答：

$$S_0 \xrightarrow{a} S_1 \xrightarrow{b} S_2 \xrightarrow{a} S_1 \xrightarrow{b} S_2 \xrightarrow{a} S_1.$$

最终在 S_1 ，不是接受状态，因此不接受。ababa 不以 abb 结尾。

题 9：三种算法的最坏复杂度分别是什么？

解答：Thompson 构造对 RE 长度 m 是 $O(m)$ 。子集构造对 NFA 状态数 n 最坏可产生 2^n 个 DFA 状态，因此最坏指数级。DFA 最小化若用朴素方法常见为 $O(k^2|\Sigma|)$ ，Hopcroft 算法可达 $O(|\Sigma|k \log k)$ ，其中 k 是 DFA 状态数。

复习总结

完整流程可以浓缩为：

1. RE 描述语言规则。
2. Thompson 算法把 RE 机械地构造成带 ϵ 转移的 NFA。
3. 子集构造用 NFA 状态集合模拟 NFA 的所有可能路径，得到 DFA。
4. DFA 最小化合并等价状态，得到更小的识别器。
5. 词法分析器基于最小 DFA 做高速扫描，并在多 token 情况下结合最长匹配和规则优先级。

理解这条链路的关键是两句话：Thompson 用 ϵ 边让构造变简单；子集构造用“状态集合”把不确定性变成确定性。